

StudyMate speaker notes

12-minute presentation with 3 minutes reserved for questions. If the teacher allows 15 minutes before Q&A, use the spare time for the live ERD and repository demonstration.

Slide 1 StudyMate

Arlan 0:00–0:40

Good afternoon. We are Study Devs, and our project is StudyMate. It helps students find people for an in-person study session. Today we will explain the problem, our requirements, the interface, and the database design. We will also show the repository and our task board. Our focus for this milestone is the plan for the system.

Slide 2 The student problem

Arlan 0:40–1:30

Our target users are university students who want to study together on campus. Imagine that you have a difficult database topic and want to practise with classmates. You need people who study the same subject, can meet at the same time, and have a suitable knowledge level. These details are often spread across separate chats. StudyMate brings the meeting details and join requests into one place. This is our problem assumption. We still need user interviews to test how often students face it.

Slide 3 The first version

Arlan 1:30-2:20

We keep the first version focused on a complete study-group journey. Students register, sign in, and create or find a group. They answer a short application form. The organizer checks the answers and the compatibility result, then accepts or rejects the application. We have left chat, shared files, ratings, and monetization outside the current version. This keeps the project realistic for our course. The interface preview has some extra ideas, but the SRS defines what we actually plan to deliver.

Slide 4 Requirements and priorities

Arlan 2:20-3:20

Our SRS contains eighteen user stories in four epics: accounts, groups, join requests, and compatibility. Each story has acceptance criteria in Given, When, Then format. There are twenty-nine scenarios in total. We used MoSCoW to separate the essential journey from improvements. For example, creating a group is Must-have, filtering by subject is Should-have, and editing saved preferences is Could-have. We do not treat a written story as an implemented feature. We use the same story identifiers in the SRS, task board, and design mapping.

Slide 5 Acceptance criteria

Agabek 3:20-4:20

Acceptance criteria tell us what we need to check. For story eleven, a complete application should create one Pending request. If the student submits again while a Pending request already exists, we should not create another one. Our non-functional requirements are measurable too. The performance target is that ninety-five percent of group-list requests finish within two seconds with one hundred concurrent users and one thousand groups. For usability, at least four out of five new students should find a group and apply within two minutes. These are future test targets. We have not claimed that the current interface passes them.

Slide 6 The interface journey

Bakdaulet 4:20-5:00

The design covers both sides of the process. The applicant opens a group, reads the meeting details, answers the questions, and submits the application. The organizer opens the incoming requests, checks the score and answers, and makes a decision. Registration, profile preferences, and group creation have separate screens. Errors appear beside the relevant fields. Pending, Accepted, and Rejected use clear text, so users do not need to understand the status from color alone.

Slide 7 Live prototype demonstration

Bakdaulet 5:00–7:00

I will now show the main journey. We open the database study session and check the subject, location, meeting time, and organizer. The applicant enters a matching subject and an available time that covers the session. Their level is different, and they leave optional preferences empty. After submission, the request is Pending. Now we switch to the organizer view. The organizer sees the answers and the seventy-percent score. They accept the request, and the applicant's status becomes Accepted. We can reset this example and show rejection as well. The prototype demonstrates navigation and intended states. It does not demonstrate real authentication or database storage.

Demo cue: pause after each click. Use the native Figma flow. Open the local HTML only as a rehearsal backup.

Slide 8 The database design

Yelnar 7:00–8:30

Our proposed database separates users, subjects, groups, and join requests. A user can organize many groups, but each group has one organizer. That is a one-to-many relationship. A student can also apply to several groups, and a group can receive applications from many students. Join requests connect these two sides and store the applicant's answers and status. In this design, group members come from Accepted requests. We do not keep a second independent membership record that could disagree with the request. The complete diagram also includes preferences and optional profile data. This is a database design for implementation, not a schema extracted from a running backend.

Slide 9 Normalization and consistency

Yelnar 8:30–9:30

We designed the schema through Third Normal Form. Subject names and course codes belong to the subject table. Groups and applications refer to a subject by its key. Multiple preferences use separate junction rows, so we avoid comma-separated values. Each descriptive attribute depends on the key of its own table. We store the submitted application answers separately from profile preferences, because a later profile edit should not change an older application. A partial unique index prevents duplicate Pending requests. Accepting or rejecting a request uses a transaction and changes only a Pending row. We also prepared a traceability table that connects interface fields with these database attributes.

Slide 10 Compatibility calculation

Yelnar 9:30–10:30

The matching rule is a simple weighted calculation. The same subject gives forty points, a compatible time gives thirty, the same knowledge level gives twenty, and matching optional preferences give ten. A component gives zero when it does not match. For the demonstration, subject and time match, while level and optional preferences do not. The result is seventy percent. We propose that compatible time means the applicant can attend the whole session. We also propose exact matching for non-empty preference sets. These details still need team agreement. The score helps the organizer compare applicants. The organizer makes the final decision.

Slide 11 Repository and team workflow

Alisher 10:30–11:30

We use GitHub for the project and Trello for tasks. The main branch contains documentation. The frontend branch contains the React interface preview, and the backend branch currently contains only a README. I will show the commit history and the Pull Request list. During our audit, the list had no Pull Requests and branch protection was disabled. These are remaining process gaps, so we cannot claim that every member has completed the required review-and-merge cycle yet. The task board contains all eighteen stories, but owners and current statuses also need updating. Our next step is for each member to review a real contribution through their own Pull Request.

Update this segment after real PRs and board corrections. Show the current evidence and do not claim the audit snapshot is still current if the team has fixed it.

Slide 12 Implementation plan

Alisher 11:30–12:00

Our proposed implementation sequence starts with accounts and shared API contracts. Next we build groups, applications, and compatibility. After integration, we check the acceptance criteria and the non-functional targets. The final weeks are for fixes and the final demonstration. We will build the eleven Must-have stories first. We are ready to discuss the design and the remaining decisions. Thank you.